SAHAYAK - CBSE Science Exhibition Guide

Simple system explanation, team roles and judge questions

Developer and project head: Johan Jyoti Kuli
Presenters: Shyamantika Pegu, Barakha Doley, Himanxhu V Ghimire

1. What SAHAYAK is

SAHAYAK is a website project that tries to recognize a limited set of Indian Sign Language (ISL) signs. A camera shows the hands. The website finds key points on the hands. A trained model uses those points to guess a sign label. The website shows the result as text. Some extra features can translate text or speak it aloud.
SAHAYAK is a prototype, which means an early version used to test an idea. It is not a complete ISL translator. It does not understand every signed sentence and must not be presented as a replacement for an interpreter.

2. Why we built it

People who use sign language may meet people who do not understand their signs. This can make daily conversations harder. A tool that recognizes some trained signs may help in limited situations. It should be tested with the people who may use it, and its limits must be explained.
The 2011 Census of India counted 5,071,007 people in its hearing-disability category. This is a historical figure, not a count of ISL users and not a current estimate. Hearing loss and using sign language are not the same thing.
Sources:
https://censusindia.gov.in/nada/index.php/catalog/43369/
https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

3. What this project shows

- A camera can help find hand key points without special gloves.
- The points can be changed into numbers.
- A trained model can compare the numbers with patterns it learned from labelled examples.
- The website can show the predicted label.
- A held pose and a moving sign need separate input paths.
- A website and a Python server can work together.
- Testing should report mistakes as well as successful examples.

4. The system in simple words

Frontend: the pages and buttons a person sees in a browser. Vercel hosts the frontend.
Backend: the Python server that receives numbers and asks the trained model to predict. Render hosts the backend.
MediaPipe Hands: software that finds key points on visible hands. In the current Live Studio, it runs in the browser.
Model: a saved file containing patterns learned from labelled examples. SAHAYAK uses scikit-learn-compatible models saved with Joblib.
API: a web address that lets the website send data to the server and receive a result.
Database: a file that can store movement samples. Saving a sample is not the same as training a model.

5. Full live-recognition workflow

1. The user opens Live Studio.
2. The browser asks for camera permission.
3. The camera provides video frames.
4. MediaPipe Hands finds up to 21 key points per hand.
5. The website draws the points on a canvas so the user can see tracking.
6. The website prepares number values from the points.
7. It sends the numbers to a Flask API on Render.

8. The backend checks the input and sends it to the correct trained model.
9. The model returns a predicted label and, when available, a confidence score.
10. The website displays the result and may add it to the message history.
11. Translation or speech may run as a separate feature and may need internet access.
The landmark drawing is only a visual guide. It is not the model's answer. Well-placed dots do not guarantee a correct prediction.

6. Static signs: why 126 numbers?

A static sign is a pose that can be described by one frame.
- Each hand has 21 key points.
- Each key point has three numbers: x, y and z.
- One hand: 21 x 3 = 63 numbers.
- Two hands: 63 x 2 = 126 numbers.
The static model expects 126 input values. This does not mean there are 126 signs or training examples. Check the live /api/status response for the real current label list.

7. Moving signs: why 1,320 numbers?

Some signs depend on movement, so one still picture is not enough.
- Each frame has 132 numbers: 126 hand-point values plus six wrist-position values.
- The system arranges the movement into 10 time steps.
- 132 x 10 = 1,320 numbers for the sequence model.
- These numbers go to a separate model for continuous signs.
This format describes movement. It does not prove every movement is recognized correctly.

8. Main software and files

- frontend/index.html and frontend/studio.html: website pages.
- frontend/studio.html: Live Studio camera and recognition workflow.
- frontend/config.js: the backend address used by the website.
- backend/web_app.py: the Flask API and its routes.
- backend/features.py: prepares static hand features.
- backend/model/sahayak_twohand.joblib: active static model.
- backend/model/sahayak_sequence.joblib: active continuous model.
- continuous_gesture_db.db and sequence_db.py: store and read movement samples in the training project where included.
- Vercel hosts website files; Render runs the backend and loads the models.
Some training scripts are in the separate local training project and may not be in the deployment-only folder. Before running a training command, check that the script exists in your local project.

9. Important API addresses

- /api/health: checks whether the server responds. It does not measure accuracy.
- /api/status: shows model status and current model labels.
- /api/predict-features: receives static numeric features from the current Live Studio workflow.
- /api/predict: an older or alternate image route. Check the current frontend before saying it is used.
- /api/sequence/predict: receives continuous-sequence features.
- /api/sequence/samples and /api/sequence/export: support movement-sample access as implemented by the backend.
- /api/translate and /api/speech: support translation or speech. Online services may need internet.

10. What the system can and cannot do

It can try to recognize signs in its active trained models and show a label. It can demonstrate one possible approach to accessibility.
It cannot understand every ISL sentence, every regional difference, facial expression, body posture, grammar or context. It can make mistakes. Lighting, camera angle, covered hands,

movement speed and differences between signers can change the result.
Do not claim perfect accuracy or full offline operation. Some drawing can happen in the browser, but live model prediction uses the hosted backend.

11. Add a static sign

1. Back up the dataset, active model and label file.
2. Use the approved collector to add the pose to data/landmarks_2hand.csv.
3. Check that the file has the new label and all old labels. Each row needs the label and values v0 through v125.
4. Train a candidate with train_model.py. It is a trial model, not the active one.
5. Check that it has 126 features and all expected labels.
6. Run python -m unittest test_sahayak.py.
7. Test fresh examples that were not used for training.
8. If the results are acceptable, run python train_model.py --promote.
9. Test new and old signs locally.
10. Back up and copy only the approved static model into the deployment backend.
11. Push the intended model file, wait for Render, check /api/status, and test the live website.

12. Add a continuous sign

1. Back up continuous_gesture_db.db and the active sequence model.
2. Use collect_sequences.py to save the full movement under one stable label.
3. Check label spelling and sample counts.
4. Train a candidate using python train_sequence_model.py --db.
5. Check that it has 1,320 features and all expected labels.
6. Record a fresh test set after training. Aim for at least five fresh examples per label.
7. Test the candidate on those fresh examples before using them for training.
8. If the results are acceptable, run python train_sequence_model.py --db --promote.
9. Run python -m unittest test_sahayak.py test_sequence.py test_sequence_db.py.
10. Test old and new signs locally.
11. Copy only sahayak_sequence.joblib to the deployment backend.
12. Push the file, wait for Render, check sequence_model_ready and sequence_sign_labels, and test the live website.

Do not replace the static model during a continuous-only update. Do not publish raw sample databases or training rows by mistake.

13. Fair testing

Keep some labelled examples separate from training. Do not test only on the examples the model learned. If possible, use different people or recording sessions for the test.
Record how many examples were tested and how many were correct for each sign. Note which signs are mixed up, and test different lighting, distances, camera angles and movement speeds. Report an accuracy percentage only when a saved test report supports it. A feature count or fast response is not an accuracy score.

14. Privacy and safe use

Ask for camera permission. Get permission from anyone whose signs are recorded for training. Explain if a frame or number is sent to a server. Keep training files safe and avoid storing identifiable video without a clear need. Do not use the prototype as a guaranteed tool for emergencies or other high-risk situations.

15. Team roles

Johan Jyoti Kuli - Lead Developer and Project Head. Full-system design and integration. He explains how the website, camera, API, models and deployment fit together.
Shyamantika Pegu - System Design, Technology and Workflow. She explains the camera-to-model steps, hand points, feature counts, backend and the two model paths.
Barakha Doley - Introduction, Problem and Project Purpose. She explains the communication

challenge, why the project was chosen, its goal and its limits.
Himanxhu V Ghimire - Live Demo, Testing and Conclusion. He demonstrates the site, explains fair testing, discusses limits and future work, and closes the talk.

16. Suggested live demonstration

1. Show the homepage and explain the goal.
2. Open Live Studio and allow camera access.
3. Show the hand points.
4. Try one sign confirmed by the current model label list.
5. Explain the predicted label and confidence carefully.
6. If possible, try another sign and an older sign.
7. Explain that one success is not proof of overall accuracy.
8. If the demo fails, show the system steps and explain the problem honestly.

17. If the live demo fails

Say: "The live result depends on camera detection and the hosted backend. We have a prototype, and this is a useful example of why reliability tests matter. I will explain the workflow and show the parts that are available while we check the connection."
Do not say a sign was recognized if no result appeared.

18. Questions judges may ask

Q: What is SAHAYAK? A: A website prototype that recognizes a limited set of trained signs.
Q: Does it translate every ISL sentence? A: No. It predicts a label from a limited set.
Q: What is a landmark? A: A key point on the hand, such as a fingertip or joint.
Q: Why 126 values? A: Two hands x 21 points x three coordinates.
Q: Why 1,320 values? A: 132 values per frame arranged into 10 time steps.
Q: Does adding a label train the model? A: No. Labelled examples, training, testing and deployment are needed.
Q: Where does the camera run? A: In the browser. The backend makes the model prediction.
Q: Does it work offline? A: Not fully. Live prediction and some extra features need internet.
Q: What is the accuracy? A: Give a number only if a fair test report supports it.
Q: What if it predicts the wrong sign? A: Record the mistake and check the camera conditions and model.
Q: Can it replace an interpreter? A: No. It is a prototype, not a guaranteed communication service.

19. Simple word list

Frontend: the part of the website a user sees.
Backend: the server that handles requests.
Landmark: a key point on the hand.
Feature: a number made from hand points.
Dataset: labelled examples used for training or testing.
Model: the saved result of training.
Prediction: the model's best guess.
API: a web address used by the website and server to exchange information.
Candidate model: a trial model that is not active yet.
Promote: approve a trial model and make it active.
Deploy: publish files to the hosted website or server.
Confidence: how strongly the model favours a label; not a promise that it is right.

20. Can GitHub train the model?

GitHub is mainly where we save and share code. Pushing a file does not automatically teach a model a new sign. Training can run on GitHub only if a specific automated workflow, training scripts, required packages, data access and enough computer resources have been set up. Check that workflow before claiming it exists. In this project, use the known local training scripts when they are present and working. Do not upload private training recordings just to make an automatic job work.

21. Change website words or design without training again

If you only change a heading, paragraph, colour, picture or page layout, you normally do not need to train a model. Make the change in the frontend files, test the page, push the intended frontend files and wait for Vercel to finish. If you change the recognition code or feature format, test carefully because the model and frontend must still agree. A new sign needs labelled data, training and evaluation; changing website text does not teach a new sign.

22. Data storage and response time

A saved sample and a trained model are different things. The continuous sample database may be a local file. Files on a hosted server may not remain after a service is replaced unless permanent storage is set up. Do not promise that collected samples will always be saved on the live server. Back up training data in a safe place and use suitable permanent storage if long-term saving is required.

The backend can record recent API response times. Those times show how long a request took in that process; they are not model accuracy and may reset when the server restarts. Measure full camera-to-screen time separately if you need that result.

23. Confidence and movement limits

The current static prediction path uses a 0.60 confidence cutoff when a probability score is available. A result below the cutoff may be treated as uncertain. This 0.60 setting does not mean the model is 60 percent accurate. The continuous-sign code can build a movement segment from about 15 to 90 frames before it predicts. These settings can change, so check the current code before giving exact values to judges.

24. Evidence to keep for testing

Keep a record of:

- Which model version was tested.
- Which signs were included.
- How many fresh examples were used for each sign.
- How many were correct and which labels were mixed up.
- Camera, lighting, distance and signer conditions.
- Any code or model change made after the test.
- The date of the live test and whether the backend was ready.

Test old and new signs after every model update. Save the real results. Do not replace missing measurements with guesses.

Final checklist

[ ] Each speaker knows their role. [ ] Current model labels checked. [ ] Camera and internet tested. [ ] No unverified accuracy number used. [ ] Old and new signs tested. [ ] Backup explanation ready. [ ] Everyone explains that SAHAYAK is a limited prototype.